



Tecnológico Nacional de México

Campus Querétaro

Título: **Entregable 11**

Facilitado por:

Julio Alejandro Villeda Maldonado I

Desarrollado por:

Ayala Sandoval Melina Dannae – 122140771@queretaro.tecnm.mx

ENTREGABLE 11 (E11)

DIAGRAMA DE CLASES UML – MODELADO ESTÁTICO COMPLETO

Unidad: Diseño

Fase: Modelado estructural del sistema

OBJETIVO

Construir el **Diagrama de Clases UML completo del sistema**, garantizando coherencia total con:

- E5 – Requisitos consolidados
- E7 – Diagrama de casos de uso
- E9 – Especificación detallada
- E10 – Diagramas de secuencia

El modelo debe reflejar correctamente:

- Entidades del dominio
- Controladores
- Servicios
- Repositorios
- Relaciones estructurales
- Atributos
- Métodos derivados de casos de uso

No se permite inventar clases sin trazabilidad.

1. Qué representa el Diagrama de Clases

El diagrama de clases representa:

- La estructura permanente del sistema.
- Las clases que lo componen.
- Sus atributos.
- Sus operaciones.
- Sus relaciones estructurales.

No representa flujo temporal.

Representa organización estructural.

2. Arquitectura Obligatoria

Se debe respetar arquitectura en capas:

Actor → Controller → Service → Repository → Entity

No debe romperse la dirección de dependencias.

Correcto:

Controller → Service

Service → Repository

Service → Entity

Incorrecto:

Entity → Controller

Repository → Controller

3. Entidades (Entity)

Qué son

Representan objetos reales del dominio.

Ejemplos:

- Usuario
 - Proyecto
 - Gasto
 - Reserva
 - Documento
 - Cliente
-

Reglas

- Derivarse de requisitos.
 - No inventar atributos.
 - No incluir lógica compleja.
 - Representar información persistente.
-

Ejemplo

```
class Reserva {  
  - id : int  
  - fecha : Date  
  - estado : String  
}
```

4. Controladores (Controller)

Función

Coordinar el caso de uso.

Responsabilidades:

1. Recibir solicitud.
 2. Validar formato básico.
 3. Invocar Service.
 4. Retornar resultado.
-

Lo que NO deben hacer

- Aplicar reglas de negocio.
 - Acceder a base de datos.
 - Contener lógica extensa.
 - Mezclar módulos diferentes.
-

Ejemplo correcto

```
class ReservaController {  
  + crearReserva(datos : ReservaDTO) : Resultado  
}
```

5. Principios de Diseño Aplicados

1 Responsabilidad Única

Un Controller solo coordina.

Si contiene:

- Cálculos
- Validaciones complejas

- Persistencia

Está mal diseñado.

2 Separación de responsabilidades

- Controller → Orquesta
- Service → Decide
- Repository → Guarda
- Entity → Representa

3 Alta cohesión

Un Controller maneja un módulo específico.

Incorrecto:

SistemaController

- registrarUsuario()
- generarReporte()
- validarPago()

4 Bajo acoplamiento

Un Controller no debe depender de múltiples Services sin justificación.

5 Derivación obligatoria

Cada método debe provenir de:

- Un caso de uso (E7)
- Un flujo detallado (E9)
- Una secuencia (E10)

Si no se puede rastrear → está inventado.

6. Servicios (Service)

Contienen reglas de negocio.

Ejemplo:

```
class ReservaService {  
    + validarDisponibilidad() : boolean  
    + confirmarReserva() : Resultado  
}
```

Aquí viven decisiones del sistema.

7. Repositorios (Repository)

Encargados de persistencia.

```
class ReservaRepository {  
    + guardar(reserva : Reserva)  
    + buscarPorId(id : int) : Reserva  
}
```

No contienen lógica de negocio.

8. Relaciones UML (Explicación Completa)

♦ Asociación

Relación estructural básica.
Usuario "1" -- "*" Reserva
Debe incluir cardinalidad.

♦ **Cardinalidades**

Opciones:

- 1
- 0..1
- 
- 1..*
- 0..*

Antes de definir cardinalidad, pregúntate:

- ¿Puede existir una instancia sin la otra?
 - ¿La relación es obligatoria?
 - ¿Puede haber múltiples instancias?
-

♦ **Composición**

Dependencia fuerte.

Reserva "1" *-- "1" Pago

Si la reserva se elimina, el pago también.

No usar composición si la parte puede existir sola.

♦ **Agregación**

Relación débil.

Empresa o-- Proyecto

El proyecto puede existir independientemente.

♦ **Herencia**

Relación "es un".

Usuario <|-- Administrador

Usuario <|-- Cliente

Usar solo cuando exista especialización real.

♦ **Dependencia entre capas**

ReservaController --> ReservaService

ReservaService --> ReservaRepository

No romper dirección.

Errores graves

- Sin cardinalidades.
 - Todas las relaciones como asociación simple.
 - Composición mal usada.
 - Entity relacionado directamente con Controller.
 - Clases aisladas sin relaciones.
-

Si en E10 aparece:

ReservaService.validarDisponibilidad()

Debe existir ese método en E11.

La estructura debe soportar el comportamiento.

10. Ejemplo Completo Integrado

@startuml

```
class Usuario {  
    - id : int  
    - nombre : String  
}
```

```
class Reserva {  
    - id : int  
    - fecha : Date  
    - estado : String  
}
```

```
class ReservaController {  
    + crearReserva() : Resultado  
}
```

```
class ReservaService {  
    + validarDisponibilidad() : boolean  
    + confirmarReserva() : Resultado  
}
```

```
class ReservaRepository {  
    + guardar() : void  
    + buscarPorId() : Reserva  
}
```

Usuario "1" -- "*" Reserva

ReservaController --> ReservaService

ReservaService --> ReservaRepository

ReservaService --> Reserva

@enduml

11. Entregables por Rol

Analista

- Lista de clases justificadas.
 - Relación requisito → clase.
 - Documento explicativo de decisiones estructurales.
-

Diseñador

- Diagrama UML completo.
- Cardinalidades correctas.
- Uso adecuado de composición/agregación.
- Separación clara por capas.

Desarrollador

- Verificación de coherencia con E10.
- Identificación de clases críticas.
- Análisis de acoplamiento.
- Análisis de cohesión.

Calidad

- Matriz requisito ↔ clase.
 - Validación de consistencia.
 - Detección de clases innecesarias.
 - Revisión cruzada E7, E9, E10 y E11.
-

MATRIZ REQUISITO

Requisito (E5)	Caso de uso (E7)	Clases Entity (E11)	Clases Controller/Service/Repository (E11)	¿Coherente?	Observaciones
RF-01 Registro de usuarios	Registrar Usuarios	Usuario	—	NO	No se observan clases para la gestión de usuarios
RF-02 Registro de transacciones	Registrar Transacciones	Transaccion, Resultado	TransaccionController, TransaccionService, TransaccionRepository	Sí	La cadena Controller→Service→Repository está definida.
RF-03 Registro de proyectos	Registrar datos del Proyecto	Proyecto	—	sí	Existe la entidad Proyecto

RF-04 Control centraliza do/panel	Acceder a panel de contro	Proyec to, Presup uesto, Gasto, Report e	PanelGastosController, PresupuestoService, PresupuestoRepository	Sí	El “panel” queda representado como PanelGastosControll er (cubre flujo de gastos/presupuesto) .
RF-05 Generació n de reportes	Generar reportes (Administ rador)	Report e	PresupuestoService / PanelGastosController	Sí	Se genera Reporte como resultado del servicio/controlador.
RF-06 Roles/per misos	Otorgar/R evocar Roles (Operado r)	—	<i>(No aparece controlador/servicio de roles en el E11 mostrado)</i>	NO	
RF-07 Gastos/ins umos y rentabilida d	Administ rar Gastos (Administ rador)	Gasto, Presup uesto, Proyec to, Report e	PanelGastosController, PresupuestoService, PresupuestoRepository	Sí	Las entidades y servicios cubren “gastos/presupuesto ” y soportan cálculo/resultado en reporte.
RF-08 Inventario	Visualizar /Editar Inventario	—	<i>(No aparece controlador/servicio de inventario)</i>	NO	El caso de uso existe en E7, pero no está modelado en el E11 del diseñador
RF-09 Proveedor es	Acceder a panel de proveedo res	Provee dor	—	PARCI AL	Existe entidad Proveedor y relación con Transaccion.

VALIDACIÓN DE CONSISTENCIA

Criterio	¿Cumple?	Observación
Controller invoca Service	Sí	Correcto en el diagrama de clases.

Service invoca Repository	SÍ	Correcto.
Entity no depende de Controller/Repository	SÍ	Correcto.
Coherencia E10 vs E11	NO	Inconsistencia: en secuencia el Controlador accede a BD (Repository) directamente. Debe pasar por Service.

REVISIÓN CRUZADA

En E10 (Diseñador) el flujo principal incluye llamadas CO → RE para acceder a base de datos

Sin embargo, en E11 (Diseñador) la arquitectura está modelada como Controller → Service → Repository

DETECCIÓN DE CLASES INNECESARIAS

Según E7, existen casos de uso para registrar Usuarios (RF-01), otorgar/Revocar Roles (RF-06), Visualizar/Editar Inventario (RF-08), Panel de proveedores (RF-09)

En el E11 del diseñador no aparecen clases para estos módulos

En E11 Analista se listan clases: Rol, Perfil, Inventario, pero en el E11 Diseñador no aparecen las clases